



Anatomy of a Neural Network

Tim Ransom



College of
**ENGINEERING, COMPUTING
AND APPLIED SCIENCES**





Outline

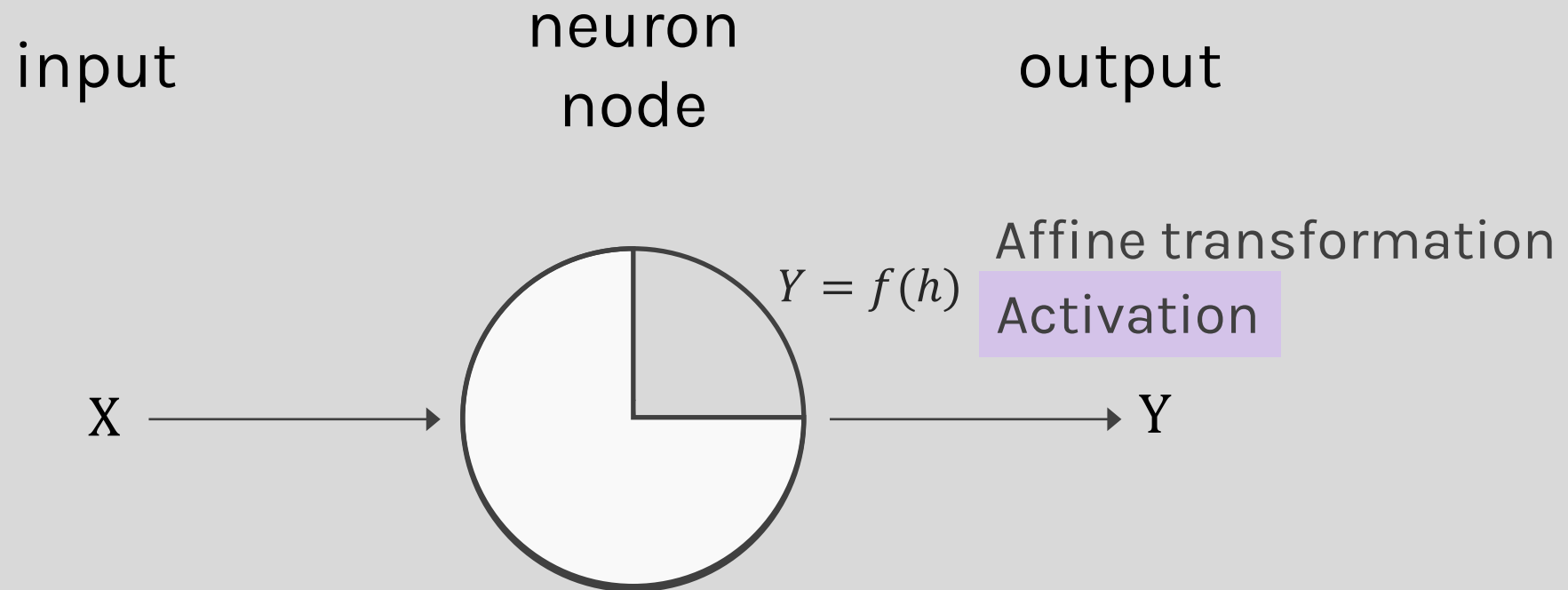
Anatomy of a NN

Design choices

- Activation function
- Loss function
- Output units
- Architecture



Anatomy of artificial neural network (ANN)



We will talk later about the choice of activation function. So far we have only talked about sigmoid as an activation function but there are other choices.

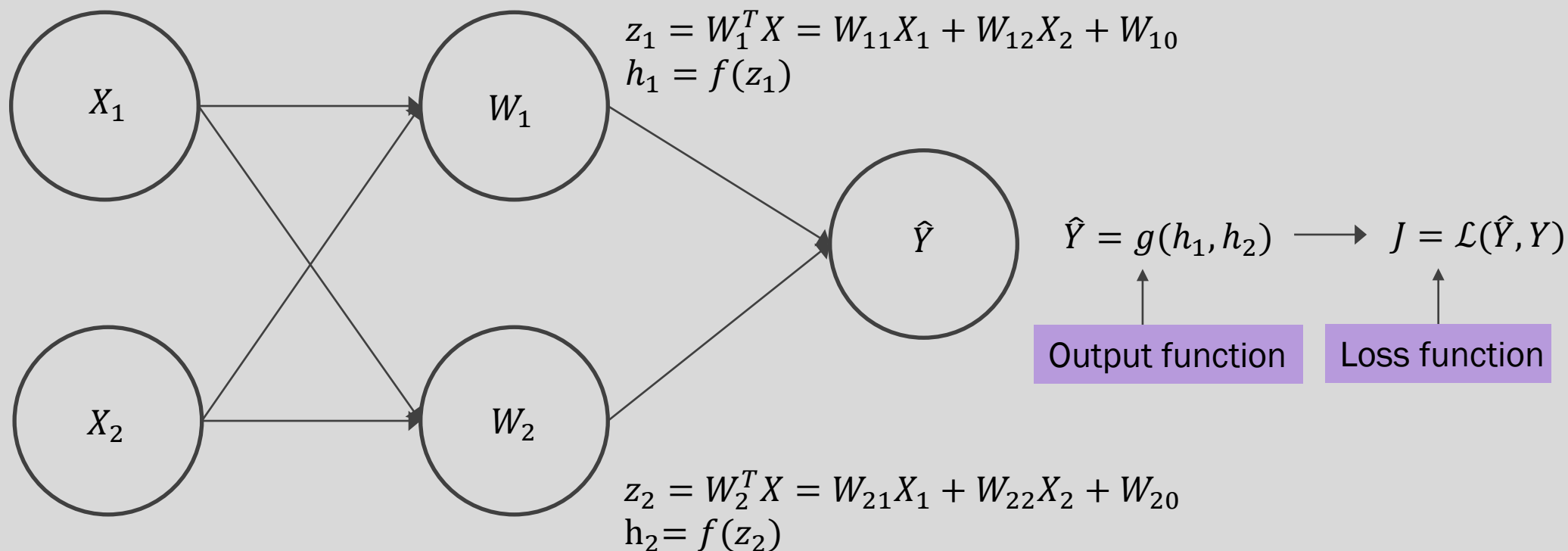


Anatomy of artificial neural network (ANN)

Input layer

hidden layer

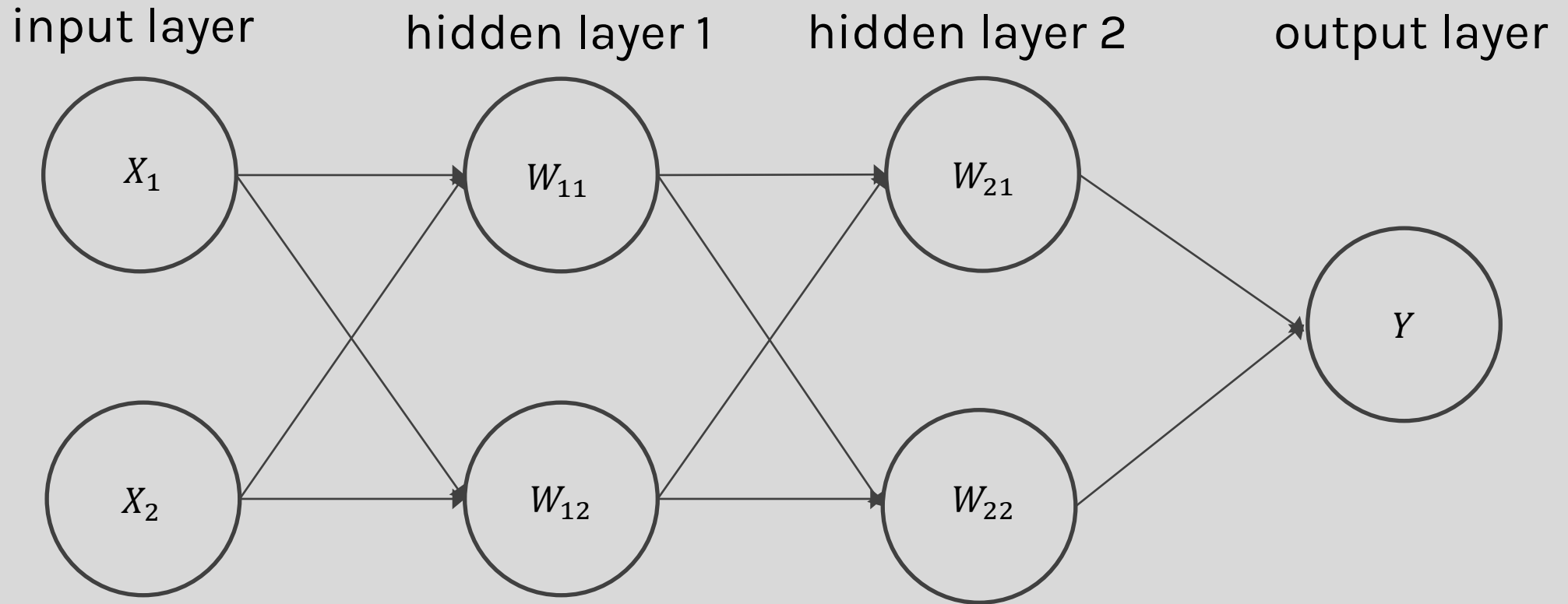
output layer



We will talk later about the choice of the output layer and the loss function. So far we consider sigmoid as the output and log-bernouli.



Anatomy of artificial neural network (ANN)





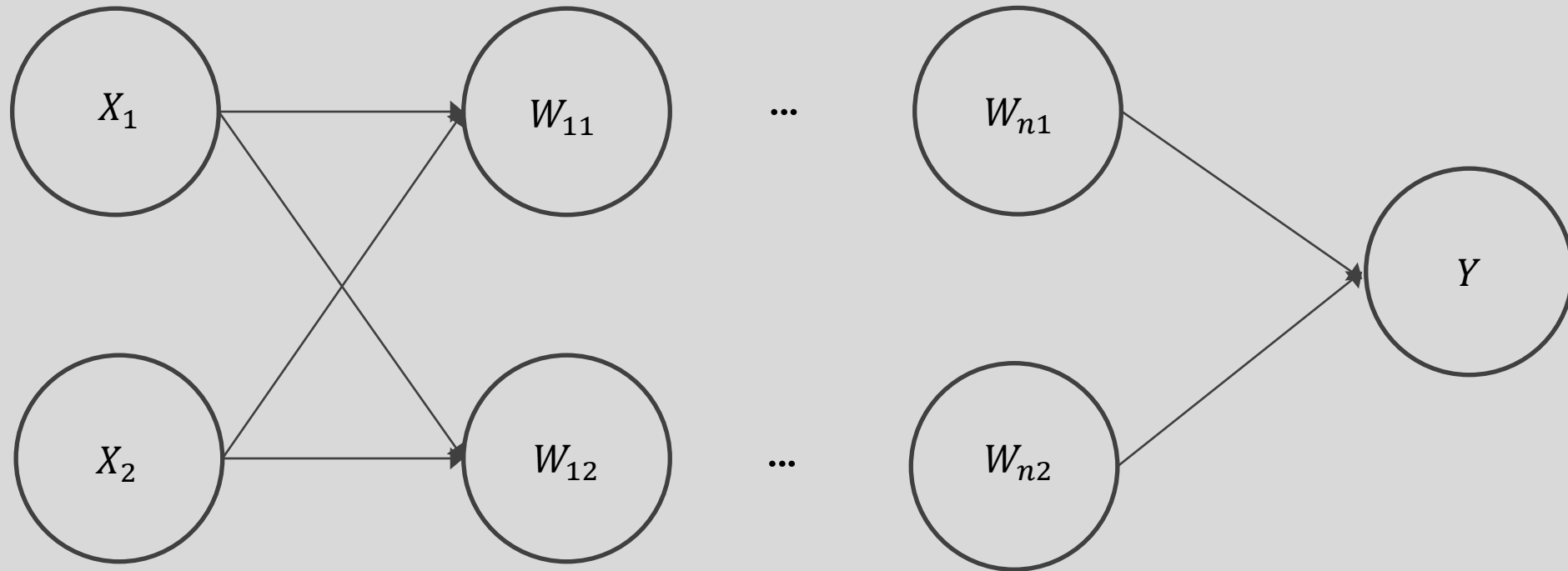
Anatomy of artificial neural network (ANN)

Input layer

hidden layer 1

hidden layer n

output layer



We will talk later about the choice of the number of layers.



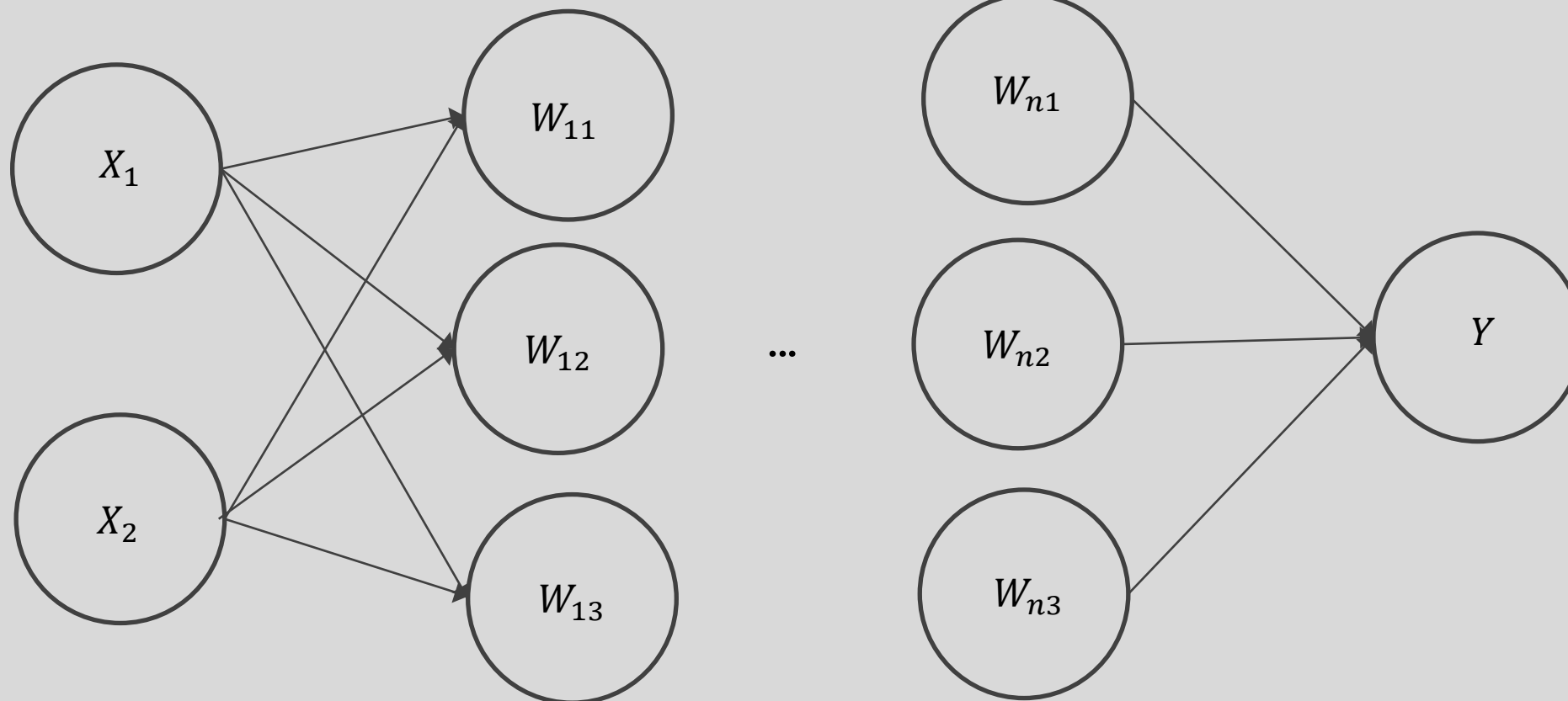
Anatomy of artificial neural network (ANN)

Input layer

hidden layer 1,
3 nodes

hidden layer n
3 nodes

output layer





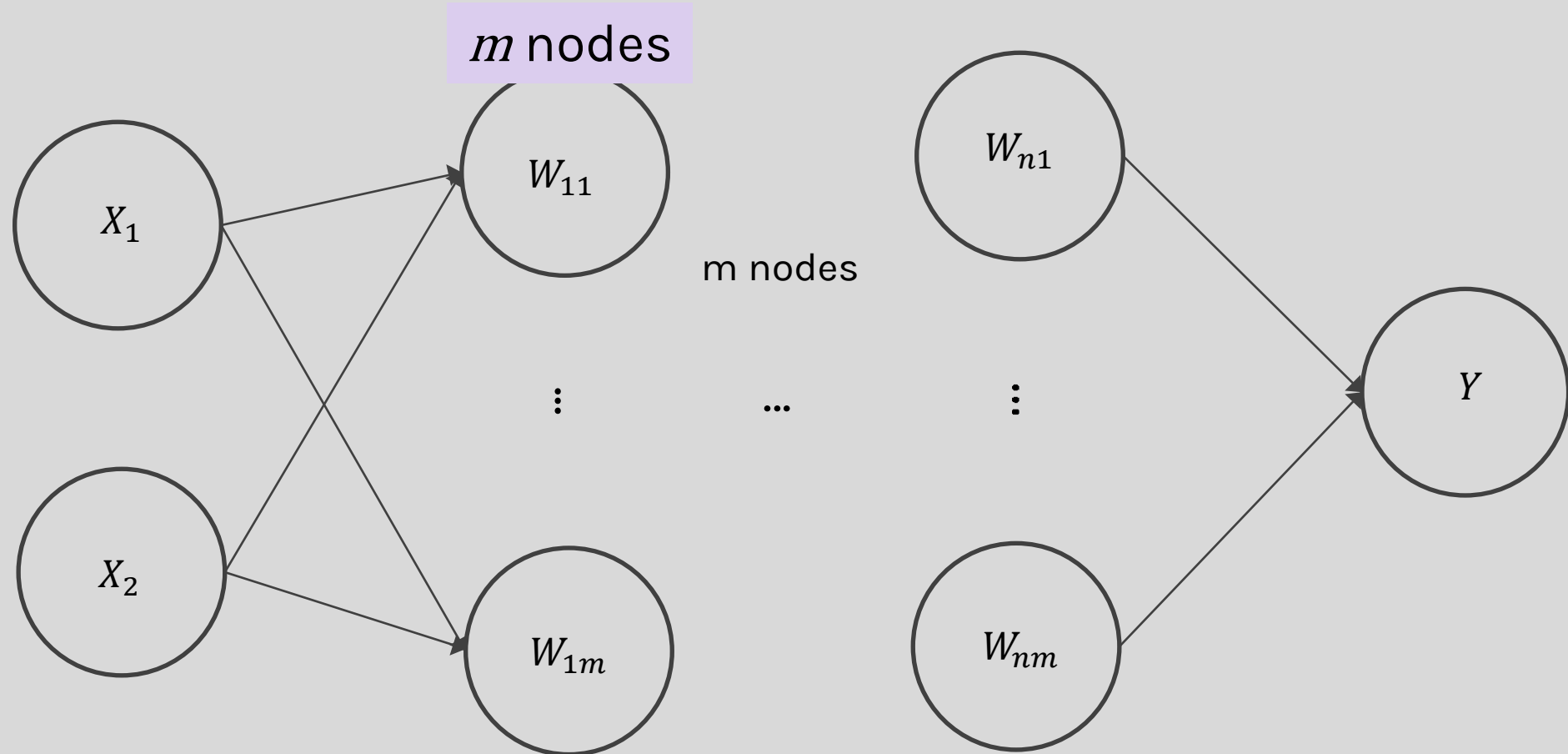
Anatomy of artificial neural network (ANN)

Input layer

hidden layer 1,

hidden layer n

output layer





Anatomy of artificial neural network (ANN)

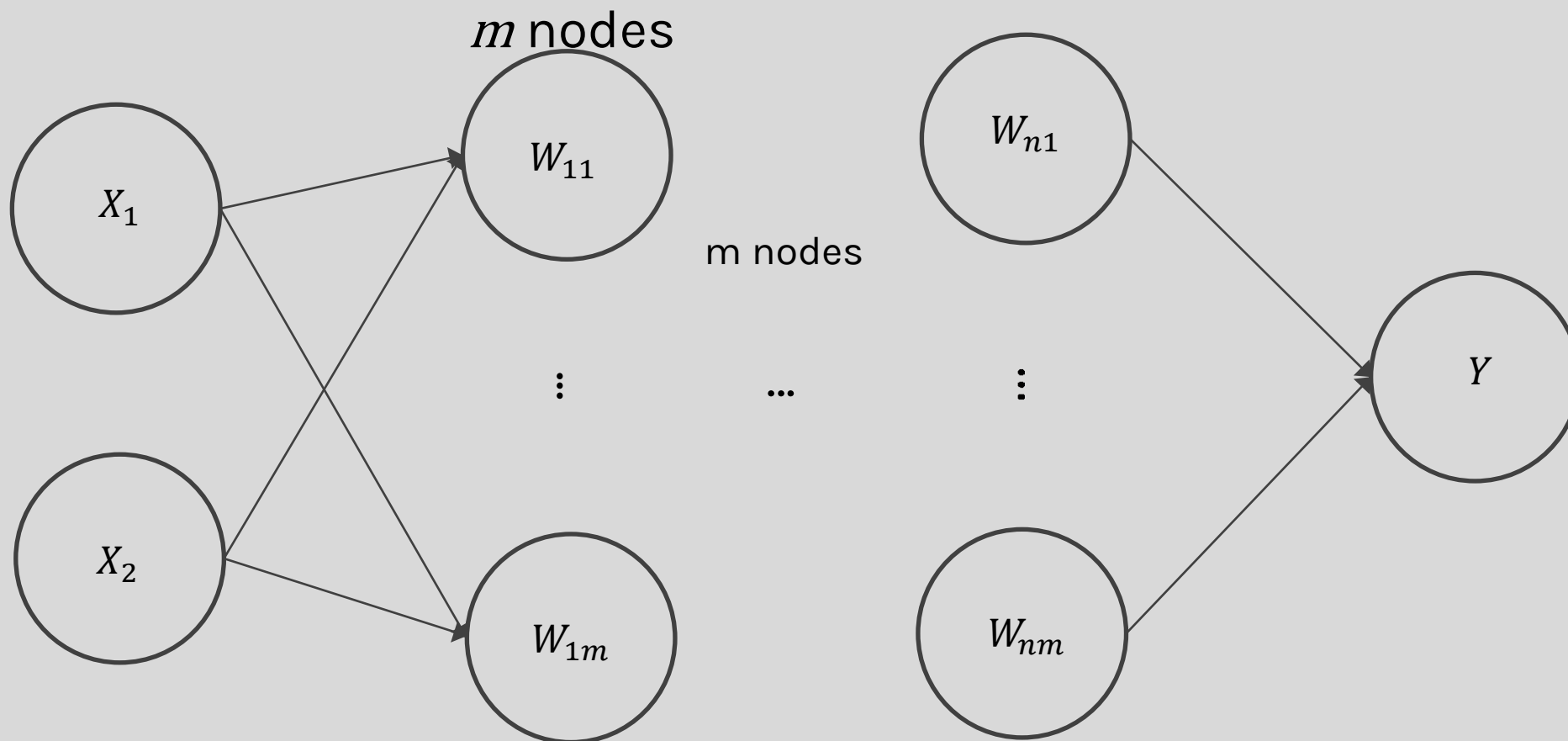
Input layer

hidden layer 1,

hidden layer n

output layer

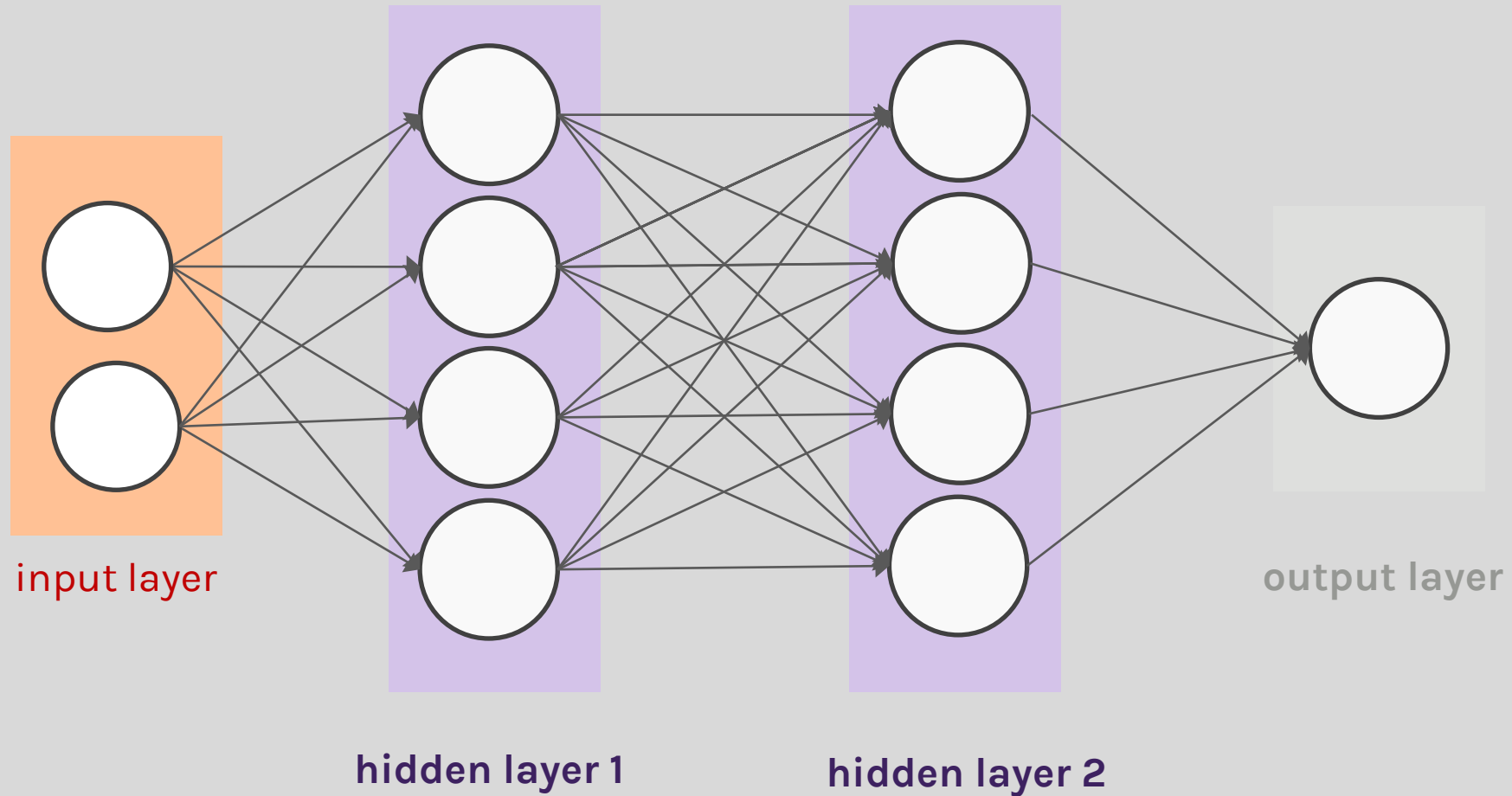
Number of inputs d



Number of inputs is specified by the data

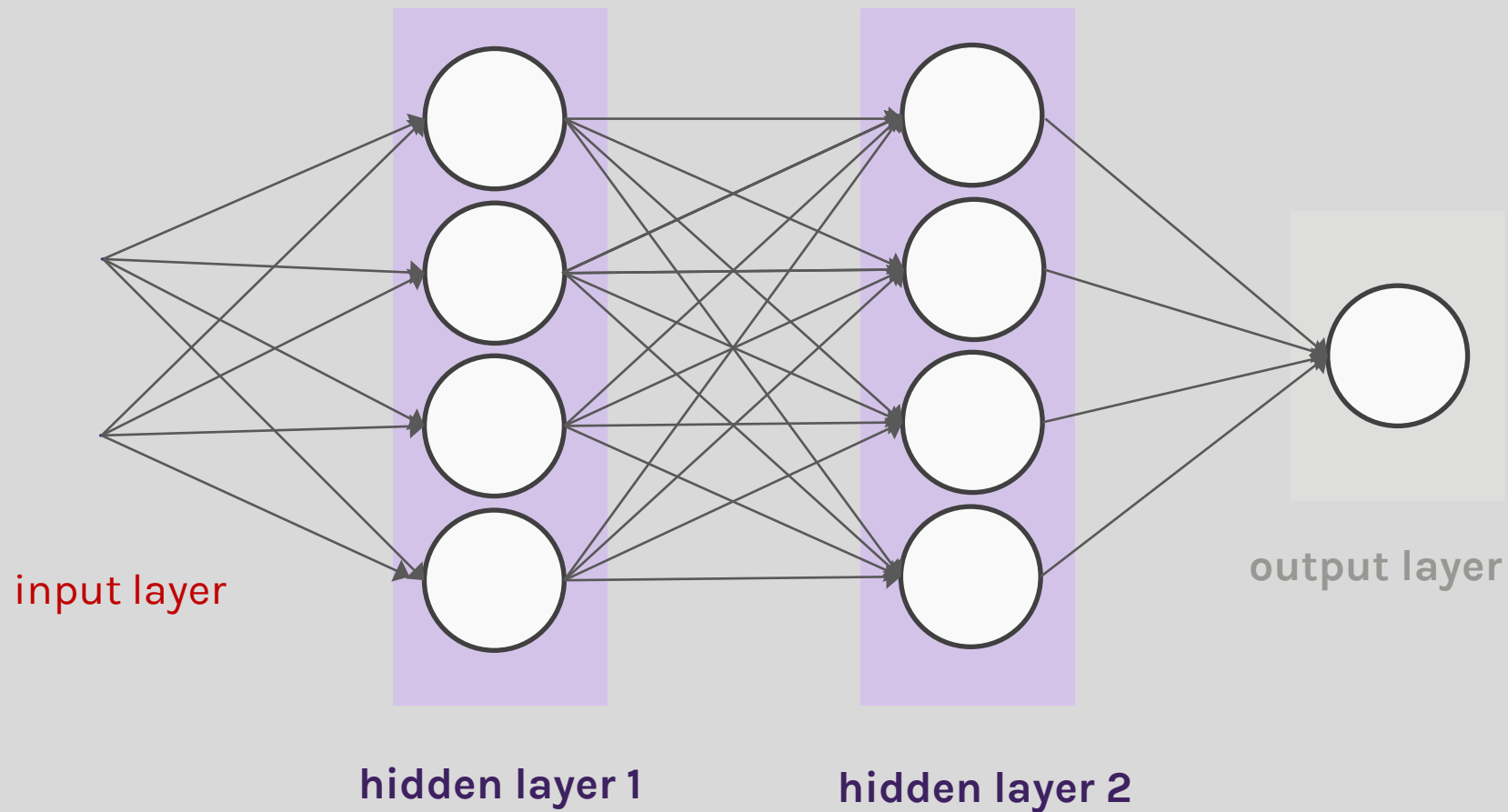


Anatomy of artificial neural network (ANN)





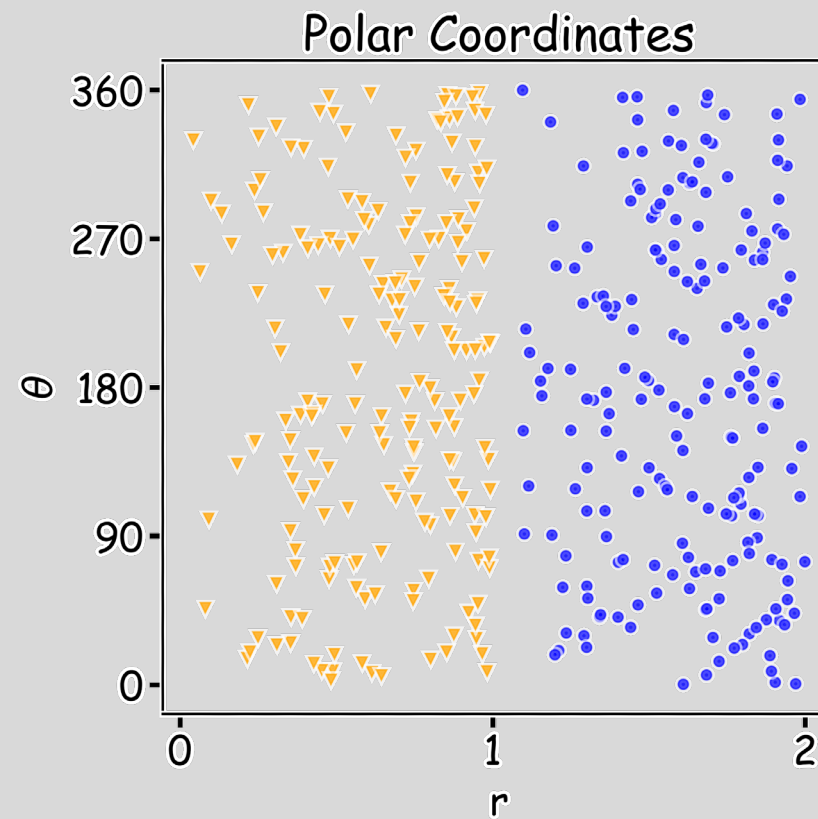
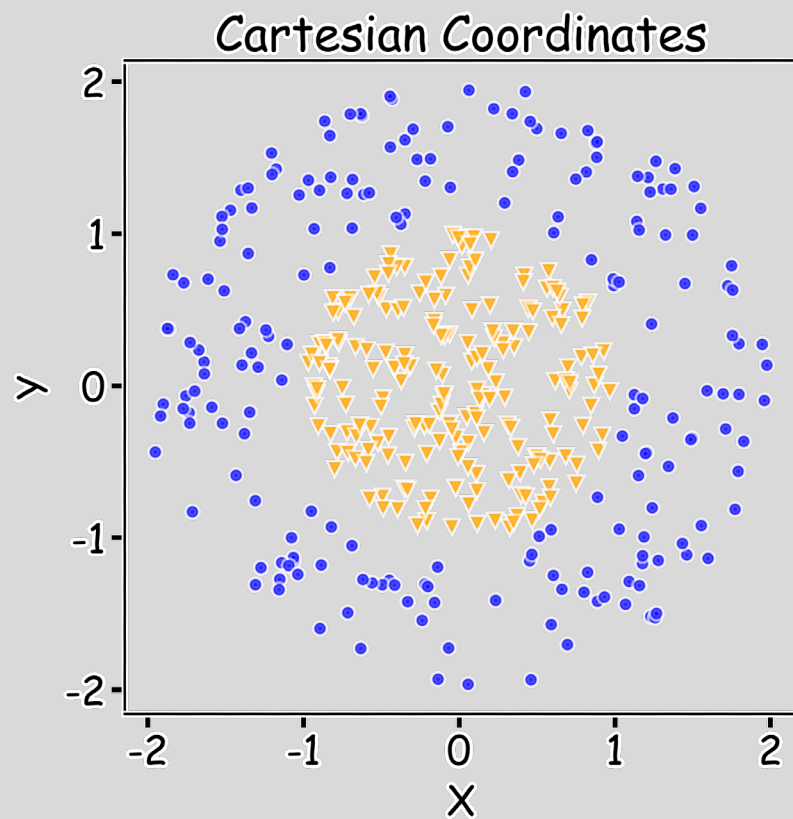
Anatomy of artificial neural network (ANN)





Why layers? Representation

- Representation matters!



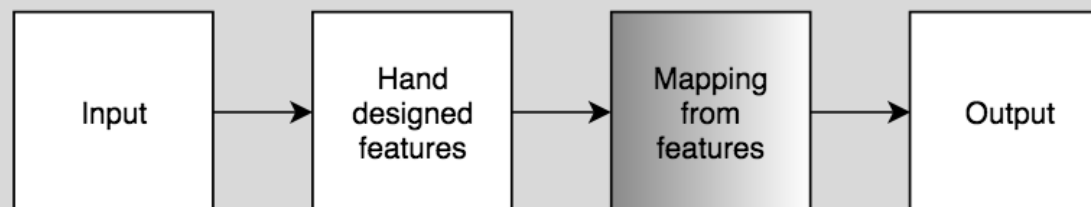


Learning Multiple Components

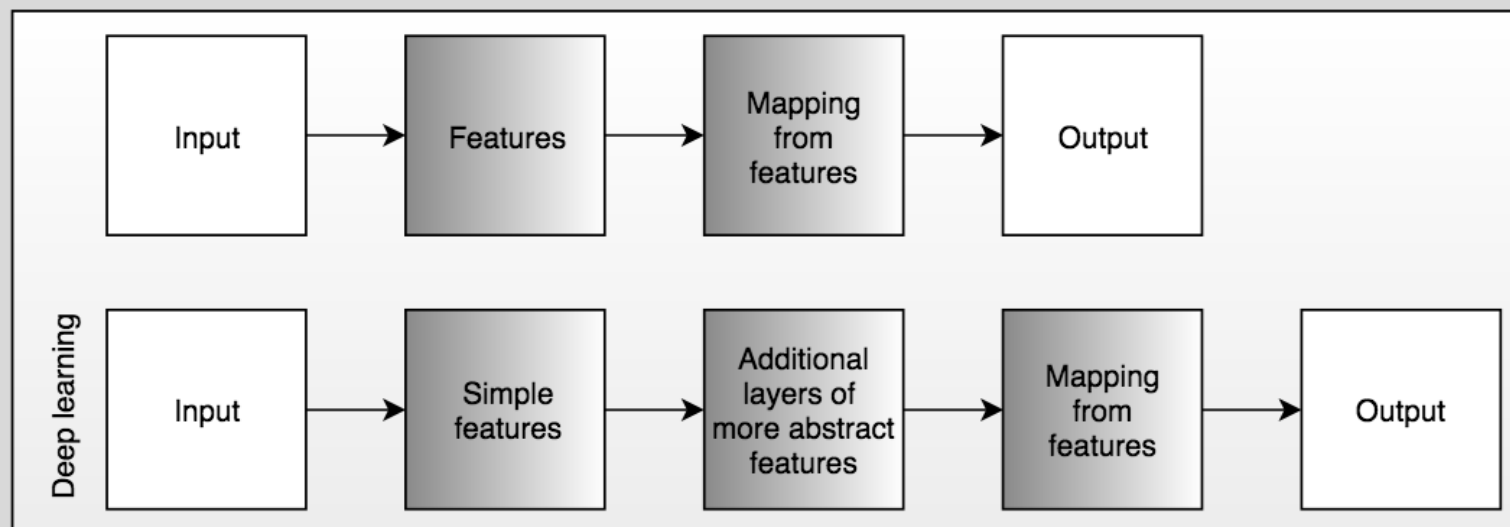
Rule-based systems



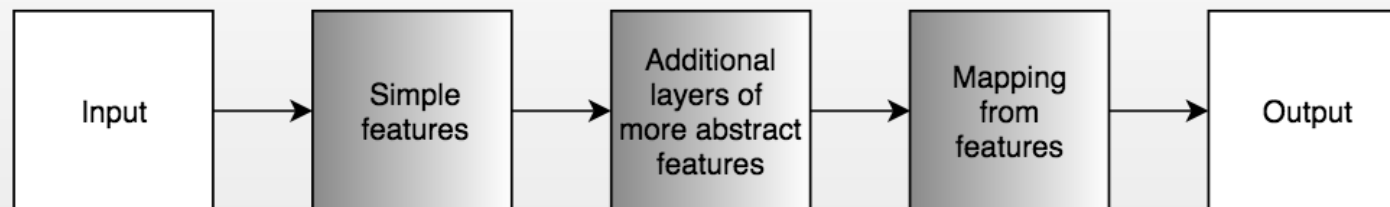
Classic machine learning



Representation learning



Deep learning





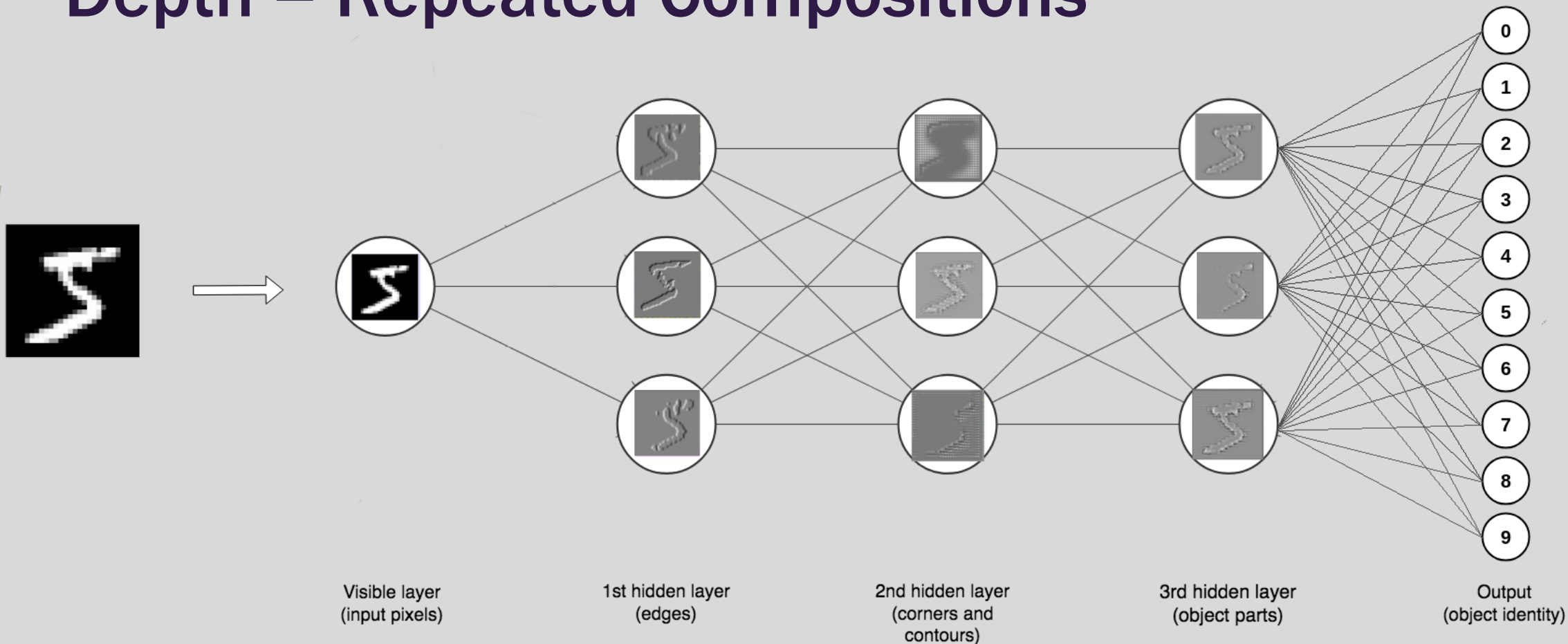
Neural Networks

Hand-written digit recognition: MNIST data





Depth = Repeated Compositions





Beyond Linear Models

Linear models:

- Can be fit efficiently (via convex optimization)
- Limited model capacity

Alternative:

$$f(x) = w^T \phi(x)$$

Where ϕ is a *non-linear transform*



Traditional ML

Manually engineer ϕ

- Domain specific, enormous human effort

Generic transform

- Maps to a higher-dimensional space
- Kernel methods: e.g. RBF kernels
- Over fitting: does not generalize well to test set
- Cannot encode enough prior information



Deep Learning

- Directly learn ϕ

$$f(x; \theta) = W^T \phi(x; \theta)$$

- $\phi(x; \theta)$ is an automatically-learned **representation** of x
- For **deep networks**, ϕ is the function learned by the **hidden layers** of the network
- θ are the learned weights
- Non-convex optimization
- Can encode prior beliefs, generalizes well